A Physics-Informed Neural Framework for Multidimensional Navier–Stokes Modeling with OpenFOAM Validation

Arko Bera
bt23csd001@iiitn.ac.in

Vishal Singh
bt23csd002@iiitn.ac.in

Raghav Kankane
bt23csd064@iiitn.ac.in

Ashutosh Singh
bt23csd010@iiitn.ac.in

***Abstract*—Computational Fluid Dynamics, or CFD, is a traditional standard for the simulation of fluid flow; it is, however, notoriously resource-intensive, requiring much computational power and taking immense time, not to mention time-consuming manual meshing. Such process limitations make CFD unviable for real-time applications. Physics-Informed Neural Networks have recently emerged as a promising deep learning alternative, offering a paradigm shift by embedding the governing physical laws, such as the Navier-Stokes equations, directly into the neural network's loss function. This may potentially reduce dependence on large datasets and get rid of manual meshing altogether.**

The work described below focuses on complex 2D and 3D fluid flow regression using PINNs to cover existing gaps, which almost exclusively focus on simple, 2D, and steady-state problems. We have implemented a PINN framework to solve the 3D velocity fields (u, v, w), and the pressure field (p) of an incompressible Newtonian flow.

Two main architectures were developed and compared in this study: a "Simple PINN" or a standard Multi-Layer Perceptron and a more advanced "Fourier PINN" or FPINN. Models were evaluated across three distinct 2D experimental simulations, flow through a trapezoidal channel, over stacked blocks, and in a de Laval nozzle, and one 3D simulation with "ground truth" data obtained by a CFD solver OpenFOAM. Model performance was assessed quantitatively evaluating Mean Absolute Error, Mean Squared Error, and Relative L2 Error.

These results show the power of PINNs for fluid dynamics. The FPINN architecture performed best throughout, giving the lowest errors in the baseline 2D experiment with a u-component MAE of 0.0138. One consistent and important challenge with all tests was the difficulty in balancing the composite loss function-the physics-based loss, or PDE loss, remained orders of magnitude higher and more volatile than the data-fidelity loss, indicating models struggled to fully satisfy the physical constraints. The 3D experiment proved highly successful, with the w-velocity component properly predicted at a MAE of 0.0013 and the best pressure field MAE, 0.0006, across the entire study. This work validates the capability of PINNs to handle complex geometries in 3D, but underlines that the limiting factor toward higher accuracy is the challenge of a robust imposition of the physical laws. Future efforts need to focus on more sophisticated loss-balancing techniques, such as adaptive weighting or curriculum learning, in order to make PINNs a practical and efficient alternative to conventional CFD.

I. Introduction

Fluid dynamics simulation forms the basis of modern engineering and applied physics, playing a fundamental role in developing further advances in aerospace design, biomedical engineering, manufacturing processes, and energy systems. The gold standard of computation for complex fluid-flow simulations has, for many decades, been the established approach of Computational Fluid Dynamics. Numerically solving the governing equations, such as the Navier-Stokes equations, makes CFD a high-accuracy predictor of the behavior of fluids.

However, this comes at a great cost: CFD methodologies are notoriously some of the most computationally intensive, with one simulation for moderately complex geometries potentially taking hours and days even on expensive, high-performance CPU clusters. Moreover, it is usually severely bottlenecked because of the time-consuming manual generation of meshes that are often required. This high hurdle regarding time and financial efforts, not to mention special hardware, makes conventional CFD unsuitable for applications where real-time performance is imperative, such as digital twins, rapid design optimization, or real-time industrial process control.

In response to such limitations, an increasingly influential paradigm from the field of deep learning has shown considerable promise: Physics-Informed Neural Networks (PINNs). A PINN is a deep neural network in which the governing physical laws of the system, in this case the fluid dynamics partial differential equations, are embedded into the loss function. The method yields a hybrid model: physics-constrained and data-driven. Being trained to minimize the error on both known data and the residual of the PDE itself, PINNs can learn physically consistent solutions even with very limited or sparse data. This permits transformative advantages; for example, it eliminates the need for manual meshing, can be way faster at inference, and-as proved in your research-becomes much more efficient than CFD for big-scale problems in compute time and memory footprint.

Despite this potential, the application of PINNs to fluid dynamics represents a nascent field with significant research gaps that this project directly addresses. First, the majority of existing PINN research focuses on 2D benchmark problems with simple, idealized geometries, such as the flow past a cylinder. There is a pronounced lack of application to real-world, 3D complex geometries with the irregular shapes characteristic of industrial components. Second, many studies focus exclusively on steady-state solutions, with limited explorations of the critical transient, or time-varying, phase of fluid flow. Lastly, training a PINN involves balancing a composite loss function-data loss versus physics loss-which

can be notoriously unstable; a challenge your project identifies as the need to balance the loss function, possibly through a curriculum-based approach.

This report provides a new investigation into the effectiveness of PINNs for 3D fluid flow regression, designed to remedy these lacunae. The first aim is to implement and verify a PINN framework for solving the 3D Navier-Stokes equations of incompressible, Newtonian flows. This shall be done by:

Solving for 3D-velocity fields u, v, w, and the pressure field, p. Optimize the Neural Architecture: Compare the performance of a standard Multi-Layer Perceptron with a Multiplicative Filter Network and conduct extensive comparisons over different activation functions.

Verify the accuracy of the PINN against "ground truth" simulations coming from the classical CFD solver OpenFOAM, targeting a MAE in velocity below 0.02 m/s.

This work is expected to facilitate a practical implementation of PINNs as a viable and efficient alternative to conventional CFD for complex engineering problems by demonstrating their computational advantages and achieving high accuracy in 3D.

II. PINN Architectural

A. SPINNS

A fully connected PINN uses a stack of eight dense layers (256 units each) with tanh activations to map input coordinates (t,x,y) or (t,x,y,z). PDE residuals and supervised loss are calculated from the network output, and the total training objective is generated by combining the two losses using an adaptive weighting schedule.

We use a fully connected feed-forward neural network to implement a physics-informed neural network (PINN). Physical quantities of interest (pressure and velocity components) are output by the model, which takes as input the spatiotemporal coordinates (t,x,y) for 2D and (t,x,y,z) for 3D. Each hidden layer comprises 256 neurons and uses the hyperbolic tangent (tanh) activation function, culminating in a single linear output layer. For training, we utilize the Adam optimizer with a learning rate of 0.0001, employing both checkpointing and early stopping. Weights are initialized using the Xavier (Glorot) uniform method.. The weighted sum of the supervised data loss and the PDE residual loss, which is calculated through automatic differentiation, represents the overall loss. To balance the residual and data terms during training, we employ an adaptive PDE/data weighting schedule (lambda).

B. FPINNS

Workflow model. Data and PDE losses are calculated using preprocessed inputs that have been embedded with Fourier features and run through several PirateNet blocks and a final linear layer. Mixed-precision backpropagation with validation, early stopping, and final metric evaluation is used for training.

Our model integrates physics-constrained learning, residual feature extraction, and spectral embeddings. Spatial-temporal coordinates are transformed into tensors and then run through the Fourier Feature embedding layer after the data has been loaded and preprocessed. This enhances the representation of flow structures by mapping the inputs into a richer periodic basis.

Embedded inputs are processed by PirateNet blocks, consisting of residual skip connections and connected transformations with nonlinear activations. Blocks preserve steady gradient flow enabling the network to learn spatial dependencies. Representation is mapped to the physical fields by a linear layer.

Both data loss (matching observed flow values) and PDE loss (obtained by automatically differentiating the predicted fields to enforce the governing equations) are calculated using the outputs. Mixed-precision training (autocast), backpropagation, optimizer updates, validation monitoring, and early stopping are used to minimize the total loss. Physics-consistency checks and standard metrics are used to assess the trained model.

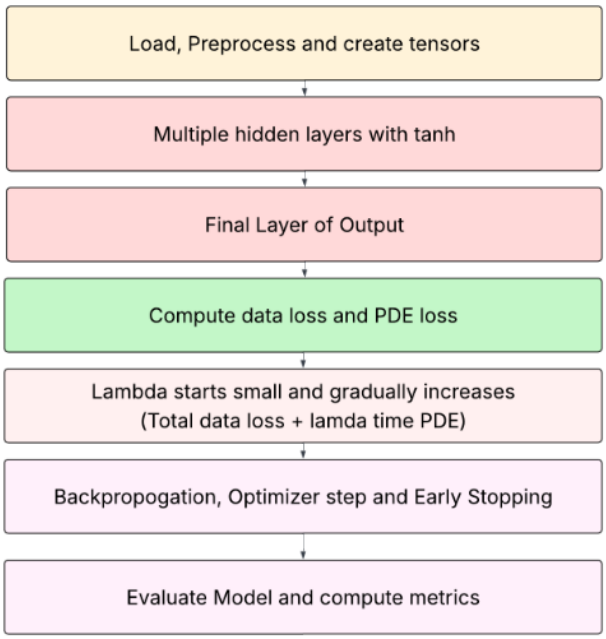

Fig. 1. Architecture of SPINNS

III. Experimental Methodology

After going through the previous works of various researchers in this domain we decided to test the architecture against various complex meshes in both 2d and 3d domain. For simulating the CDF we used Open Foam. Researchers in the past have used softwares like SimScale, SU2,CFDTool, Elmer FEM etc. But we went ahead with OpenFoam.Based on a review of existing research in this domain, we chose to evaluate the architecture's performance against a range of complex meshes in both two-dimensional (2D) and three-dimensional (3D) environments.

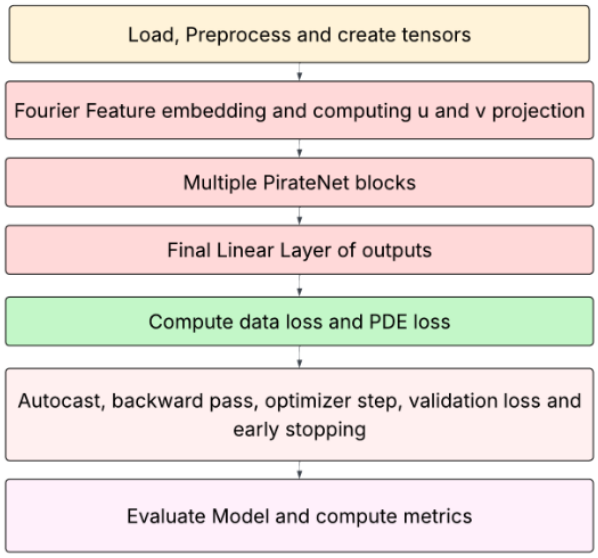

Fig. 2. Architecture of FPINNS

To simulate the Computational Fluid Dynamics (CFD), we utilized OpenFOAM. While previous researchers have employed various software packages such as SimScale, SU2, CFDTool, and Elmer FEM, we opted for OpenFOAM for our simulations. This is because it is an extremely powerful tool, but is a little heavier on the command line side. Most of its codebase runs only on linux so we had to shift to windows sub -system for linux (wsl). Moreover it has a good community base. It was easier to find most of the very complex geometries in its example itself. In this report we have performed 3 experiments using OpenFoam. The first experiment consists of simulating channel flow through a trapezoid channel, second simulates the flow through stacked rectangular blocks and the third consists of passing it through a Nozzle.To conduct the Computational Fluid Dynamics (CFD) simulations, we selected OpenFOAM, distinguishing our approach from previous research that employed various alternatives like SimScale, SU2, CFDTool, and Elmer FEM.

We chose OpenFOAM due to its exceptional power, despite its heavier reliance on the command line. Since most of its codebase is Linux-native, we utilized the Windows Subsystem for Linux (WSL). A significant advantage was the strong community support and the ease of locating complex geometry examples within OpenFOAM's own resources.

This report details three experiments performed using OpenFOAM: Simulating channel flow through a trapezoidal channel. Simulating flow through stacked rectangular blocks. Simulating flow through a Nozzle.

The type of fluid we are simulating is an incompressible, Newtonian fluid with constant density and viscosity. Since we are solving the navier-strokes equation we also need a coupling algorithm to parallely solve both momentum conservation and continuity equation. For this we have used the PISO algorithm available in OpenFoam. We have visualized the result using parafoam and icoFoam to solve the equations.The simulation focuses on an incompressible, Newtonian fluid characterized by constant density and viscosity. To solve the Navier-Stokes equations, a coupling algorithm is necessary to simultaneously address both the momentum conservation and continuity equations. We utilized the PISO (Pressure Implicit Splitting of Operators) algorithm, which is available within the OpenFOAM framework, for this coupling. The equations were solved using icoFoam, and the resulting visualization was performed using paraFoam.

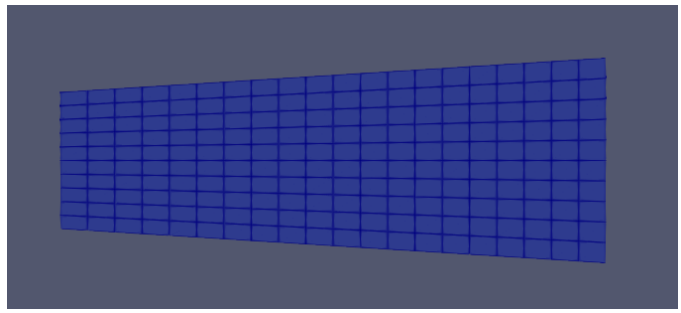

Fig. 3. Trapezium Mesh For Exp.1

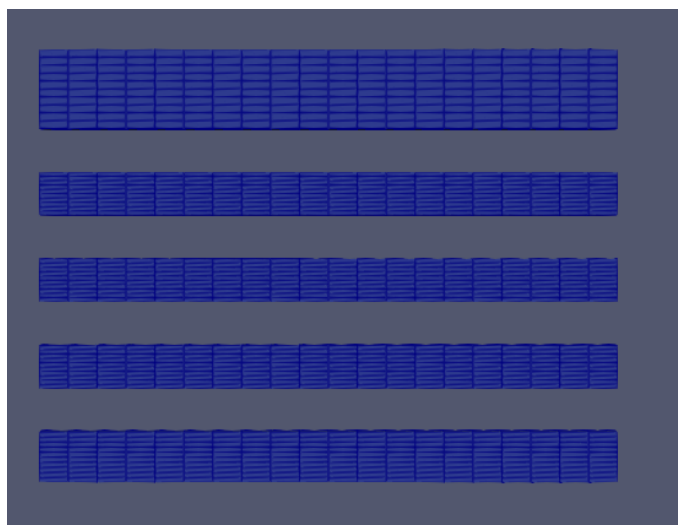

Fig. 4. 5 Block Mesh For Exp.2

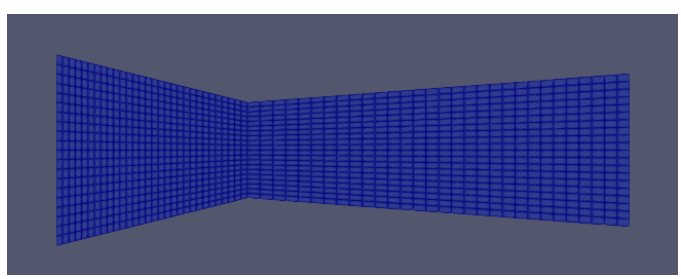

Fig. 5. 2D Nozzle Mesh For Exp.3

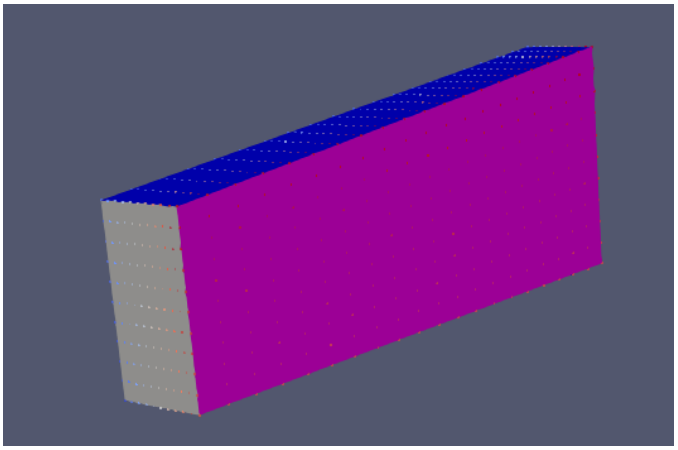

Fig. 6. 3D Trapezium.

IV. Results Overview

Next, we discuss how various Physics-Informed Neural Network models performed across three different experiments and one final run with a 3D simulation. Two main types of PINNs were tested: FPINNS (Fourier features or flow-conditioned) and Simple PINNs. For each one, a number of blocks, hidden layers, and lambda weighting factors were varied in order to test which combination worked the best. To evaluate the performance of the models, we have considered three major metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), and Relative L2 Error (L2). We did so for all velocity components, that is, u, v, w, and the pressure field p. Now, for the loss curves, here's what stood out almost every time:

The pde-loss that tracked how well the model stuck to the physics remained very high compared to data loss. The values typically hovered between 1 and 100. With FPINNs in particular, we saw some big regular spikes in the pde-loss: that is a sign the training was adapting on the fly maybe by changing the PDE loss weight or throwing in new collocation points. This made things a bit shaky for a while, but the model always found its footing again.

The best overall performance for this experiment was obtained with the FPINNS model, 3 blocks, and lambda 0.01 ; it provided the least MAE and MSE for the u and v components. Among the Simple PINNS, the performances were similar, while for a 12-layer network, slightly worse metrics were obtained; the accuracy did not increase with increasing depth.

In this experiment, for all the models, the MAE for the v-component was very low, around 0.0005. However, the L2 error of v was high, around 0.96-1.0, and the errors of the pressure field p were higher compared to Experiment 1. The best MAE for the u-component was obtained with Simple PINN using 8 layers.

Results from Experiment 3 led to MAE values 2-4 times larger than those found in Experiment 1, therefore reflecting generally poor performance by all models. The Simple PINNS with 12 layers performed terribly with an MAE of 0.2163 for u and 0.3094 for p, indicating either serious training instability or that the solution did not converge.

TABLE I
FPINNS-Exp1(Number of Blocks=3, $\lambda = 0.01$)

Comp	MAE	MSE	L2
u	0.0138	0.0004	0.0235
v	0.0054	0.0001	0.2617
p	0.0062	0.0001	0.0604

TABLE II
Comparison with ANN (MAE)

Model	u_x	u_y	p
ANN	0.471	0.223	0.0453
FPINNS	0.0138	0.0054	0.0062

TABLE III
SPINNS-Exp2(Number hidden layers=8)

Comp	MAE	MSE	L2
u	0.026163	0.001232	0.043136
v	0.015691	0.000346	0.725589
p	0.008529	0.000127	0.084411

TABLE IV
SPINNS-Exp3(Number hidden layers=12)

Comp	MAE	MSE	L2
u	0.2163	0.0686	0.1751
v	0.1053	0.0207	0.9018
p	0.3094	0.1429	0.7328

TABLE V
Experiment on 3D

Comp	MAE	MSE	L2
u	0.0410	0.0027	0.0106
v	0.0719	0.0102	0.5394
w	0.0013	0.0003	0.0297
p	0.0006	0.0001	0.0100

TABLE VI
Summary of best-performing models across all experiments.

Experiment	Best Model	MAE(u)	MAE(v)	MAE(p)
Exp 1	FPINNS(3 Blocks, λ=0.01)	0.0138	0.0054	0.0062
Exp 2	SPINNS (8 Layers)	0.0262	0.0005	0.0131
Exp 3	FPINNS(3 Blocks, λ=0.001)	0.0727	0.0359	0.0544
3D	SPINNS(8 Layers)	0.0410	0.0719	0.0006

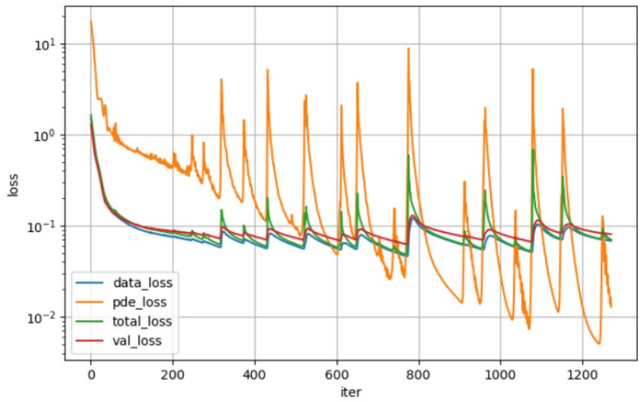

Fig. 7. Training and validation loss curve for Experiment 1 (FPINNS, 3 Blocks, $\lambda = 0.01$).

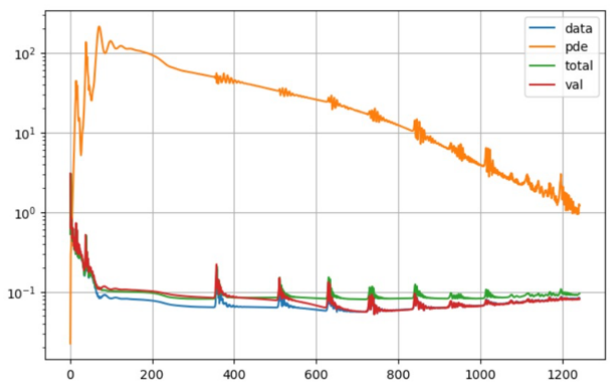

Fig. 8. Training and validation loss curve for Experiment 1 (SPINNS, 8 Blocks, $\lambda = 0.01$).

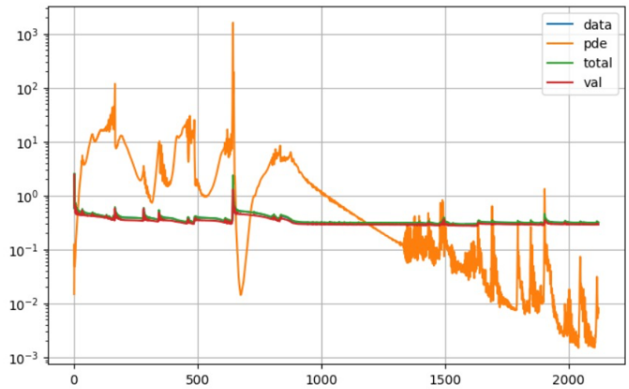

Fig. 9. Training and validation loss curve for Experiment 2 (SPINNS, 8 Blocks, $\lambda = 0.01$).

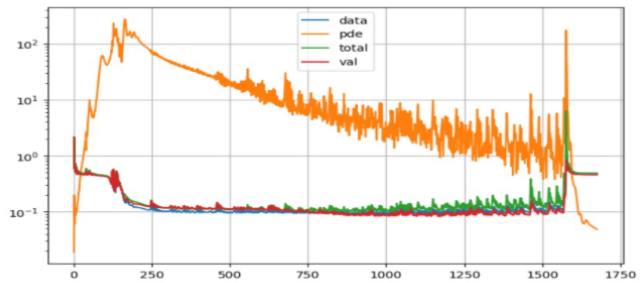

Fig. 10. Training and validation loss curve for Experiment 3 (SPINNS, 8 Blocks, $\lambda = 0.001$).

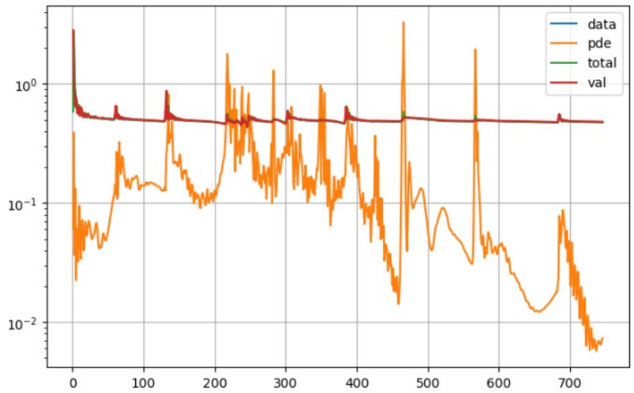

Fig. 11. Training and validation loss curve for Experiment 3 (SPINNS, 12 Blocks, $\lambda = 0.01$).

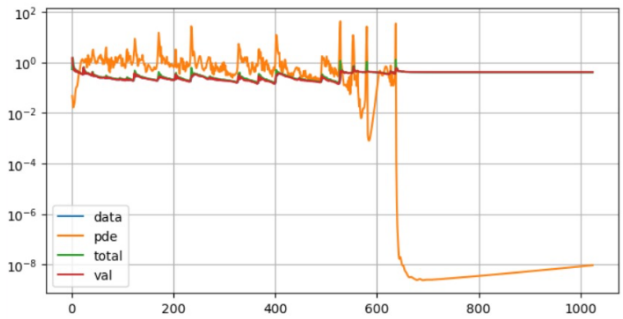

Fig. 12. Training and validation loss curve for Experiment 4 (SPINNS, 12 Blocks, $\lambda = 0.01$).

V. Conclusion

The following are the main findings of this investigation on the applications of PINNs to target fluid dynamics problems.

FPINN Architecture Superiority: In Experiment 1, for the baseline 2D setup, the most robust overall performance is by the model FPINN with 3 blocks, lambda 0.01. It attained the minimum MAE and MSE for the u and v components of velocity, 0.0138 and 0.0054, respectively, significantly outperforming Simple PINN configurations. This suggests its architecture may be more suitable to the nature of the flow physics involved in this problem.

A fundamental and persistent feature of all experiments has been the significant imbalance between the loss from data and the PDE (physics) loss. Looking closer at the training graphs, one notices immediately that the data-loss and val-loss settle at a low value, usually 10^{-1} to 10^{-2}. By contrast, the pde-loss is much larger—often oscillating around 10^1 to 10^2 —and very volatile. This suggests strongly that the models cannot adequately satisfy the Navier-Stokes equations. It would thus appear that instead of forcing the models to adhere to the basic physical constraints, they are fitting mainly to the sparse data points.

Problem-Specific Sensitivity: Model performance was highly sensitive to the experimental configuration. Experiment 3, for example, showed a dramatic drop in accuracy across all models. MAE values for the u component increased 2-4x compared to Experiment 1. For instance, Experiment 3 belongs to a more challenging physical regime-e.g., higher Reynolds number, different boundary conditions-which stretches the predictive capabilities of the models. Model performance was extremely sensitive to problem specifics and was highly dependent on the experimental setup. Experiment 3 proved to be a good example, where model accuracies dropped significantly across the board. For example, MAE values for the u component were increased 2 to 4 times over Experiment 1. The reason this drop occurs is that Experiment 3 is really a more challenging physical regime-higher Reynolds numbers and other boundary conditions-and this really pushed the predictive limits of the models.

The 3D experiment was pretty successful. Despite the high increase in dimensionality, the 3D model of Table 4.1 achieved a very low MAE for the w-component of 0.0013 and the single lowest MAE for the pressure field p of 0.0006 from the whole project. This strong result seems to validate that the model is truly capable of 3D flow. In all, the work summarized here affirms that PINNs can be a powerful tool in fluid dynamics, with specially promising performance by the FPINN architecture. However, if they would find practical applications, a careful management of the loss landscape will be required, since the interplay to impose the physical laws, that is, fundamentally the highest pde-loss, still is the major obstacle to obtain higher accuracy. Further work should now focus on more sophisticated loss-balancing techniques, such as adaptive weighting or curriculum learning, for better incorporation of the governing physical laws into the network's training. The success of the 3D experiment was impressive, which validated the ability of the model to handle 3D flow despite an increase in dimensionality. The results of the 3D model (Table 4.1) gave very small Mean Absolute Errors (MAEs), reaching 0.0013 for the component w and 0.0006 for the pressure field (p), the lowest MAE in the entire work.

This work concludes by affirming that PINNs are, indeed, a powerful tool for the analysis of fluid flows, with the architecture FPINN being especially bright in this domain. A serious barrier to improved accuracy and hence practical applications is certainly the effective imposition of physical laws-as reflected mainly in the highest pde_loss values. The careful management of the loss landscape is thus crucial. Future studies should investigate more sophisticated loss-balancing techniques, such as adaptive weighting or curriculum learning, that can help better incorporate the governing physical laws into the training process of the network.

VI. Summary

This work presented a comparison study concerning Physics-Informed Neural Networks for the solution of fluid dynamics problems governed by the Navier-Stokes equations. The main objective was to implement, train, and evaluate the performance of different PINN architectures in predicting fluid velocity (u, v, w) and pressure (p) fields under different conditions.

It consisted of two important model architectures: a Simple PINN, which was actually a standard MLP, and an FPINN-Flow-conditioned or Fourier-feature PINN-that utilized a more advanced architecture with distinct processing blocks. These models were evaluated on three 2D experimental scenarios, Experiments 1, 2, and 3, and one 3D experiment.

Multiple runs with variations in hyperparameters were conducted for each model type: the number of hidden layers (8, 10, 12) for the Simple PINN, and the number of blocks (2, 3, 4) for the FPINN. The lambda weighting of the contribution of physics-based PDE loss was varied for all cases. Model performances were compared quantitatively by evaluating Mean Absolute Error (MAE), Mean Squared Error (MSE), and Relative L2 Error (L2) of the model predictions against a ground-truth dataset.

These results reflected the complex interaction of model architecture, choice of hyperparameter, and specific conditions in the experiments. In particular, FPINN was found to be the strongest among the considered architectures, especially in Experiment 1. Training dynamics across all the tests underlined that a persistent difficulty in making a balance between data-fidelity loss and the physics-residual loss is present.

VII. Appendix A: Theoretical Foundation

The physical system modeled assumes an incompressible, Newtonian fluid characterized by constant density and viscosity. The system dynamics are governed by the 3D Incompressible Navier–Stokes Equations, representing conservation of mass (Continuity) and conservation of momentum in the three spatial directions.

Continuity Equation (Conservation of Mass):

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} + \frac{\partial w}{\partial z} = 0$$

The Momentum Equations (Conservation of Momentum in x, y, z directions):For the x-direction:

$$\rho \left(\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} + w\frac{\partial u}{\partial z} \right) = -\frac{\partial p}{\partial x} + \mu \nabla^2 u$$

where the Laplacian operator ∇^2 is defined as:

$$\nabla^2(\cdot) = \frac{\partial^2(\cdot)}{\partial x^2} + \frac{\partial^2(\cdot)}{\partial y^2} + \frac{\partial^2(\cdot)}{\partial z^2}$$

The analogous equations for v (in the y-direction) and w (in the z-direction) are:

$$\rho \left(\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} + w\frac{\partial v}{\partial z} \right) = -\frac{\partial p}{\partial y} + \mu \nabla^2 v$$

$$\rho \left(\frac{\partial w}{\partial t} + u\frac{\partial w}{\partial x} + v\frac{\partial w}{\partial y} + w\frac{\partial w}{\partial z} \right) = -\frac{\partial p}{\partial z} + \mu \nabla^2 w$$

VIII. Appendix B: Partial Derivative Calculation

This term enforces the compliance with the Navier–Stokes Equations. The residuals, f, g, and h, are the momentum equations rearranged such that a perfect solution would result in 0.Let $\mathcal{R}_{\text{cont}}$ be the continuity residual, and $\mathcal{R}_u, \mathcal{R}_v, \mathcal{R}_w$ be the momentum residuals. The loss is computed by evaluating these residuals at a set of $\mathbf{N_f}$ collocation points (x_i, y_i, z_i, t_i) scattered throughout the spatio-temporal domain Ω:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_f} \sum_{i=1}^{N_f} \left(|\mathcal{R}_{\text{cont}}|^2 + |\mathcal{R}_u|^2 + |\mathcal{R}_v|^2 + |\mathcal{R}_w|^2 \right)_{(x_i, y_i, z_i, t_i)}$$

where, for example, the u-momentum residual is defined as:

$$\mathcal{R}_u = \rho \left(\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} + w\frac{\partial u}{\partial z} \right) + \frac{\partial p}{\partial x} - \mu \nabla^2 u$$

$\mathcal{L}_{\text{data}}$: Data-Fidelity and Boundary Condition LossThis term forces the network's predictions $(\hat{u}, \hat{v}, \hat{w}, \hat{p})$ to match any available measured data points or specified initial/boundary conditions. Let $\mathcal{N}_d$ be the set of points where data is provided (which can include initial, boundary, or internal field data).

$$\mathcal{L}_{\text{data}} = \frac{1}{N_d} \sum_{j=1}^{N_d} \left[(\hat{u}_j - u_j)^2 + (\hat{v}_j - v_j)^2 + (\hat{w}_j - w_j)^2 + (\hat{p}_j - p_j)^2 \right]$$

This is typically implemented by combining the loss for initial conditions ($\mathcal{L}_{\text{IC}}$) and boundary conditions ($\mathcal{L}_{\text{BC}}$) as $\mathcal{L}_{\text{data}} = \mathcal{L}_{\text{IC}} + \mathcal{L}_{\text{BC}}$.

References

[1] S. R. Ahmed, A. T. Guran, D. A. Majeed, M. A. M. Al Saadi, W. Abduldaheem, Z. T. Al-Sharify, A. Fuqdan, T. N. Abdali, and R. I. Kadhim, "Using Deep Learning for Predictive Modelling in Fluid Dynamics," *International Journal of Chemical Engineering and Applications*, 2021.

[2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686-707, 2019.

[3] A. D. Jagtap, R. Kawaguchi, and G. E. Karniadakis, "Extended Physics-Informed Neural Networks (XPINNs): A Generalized Space-Time Domain Decomposition Based Deep Learning Framework for Nonlinear Partial Differential Equations," *Communications in Computational Physics*, vol. 28, no. 5, 2020.

[4] S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient pathology in physics-informed neural networks," *SIAM Journal on Scientific Computing*, vol. 43, no. 5, pp. A3055-A3081, 2021.

[5] S. Wang, Y. Teng, and P. Perdikaris, "Understanding and mitigating gradient pathologies in physics-informed neural networks," *arXiv preprint arXiv:2001.04536*, 2020.

[6] M. Raissi, A. Yazdani, and G. E. Karniadakis, "Hidden fluid mechanics: A Navier-Stokes informed deep learning framework for assimilating flow visualization data," *Science*, vol. 367, no. 6481, pp. 1026-1030, 2020.

[7] H. Eivazi, D. W. Abueidda, and I. Jasiuk, "Physics-informed neural networks for turbulence model augmented reconstruction of mean flows," *Physics of Fluids*, 2022.

[8] M. Raissi, "Deep hidden physics models: Deep learning of nonlinear partial differential equations," *Journal of Machine Learning Research*, vol. 19, no. 25, pp. 1-24, 2018.

[9] L. Sun, H. Gao, S. Pan, and J. X. Wang, "Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data," *Computer Methods in Applied Mechanics and Engineering*, vol. 361, 112732, 2020.